

Phase 10B - limitations

- [PHASE 10B — LIMITATIONS](#)
 - [1. No live Gemini API call was made in this build sandbox](#)
 - [2. The fidelity validator checks a fixed, explicit list of protected fields](#)
 - [3. Contradiction detection depends on recognizable labeling](#)
 - [4. Currency-representation matching covers common forms, not all possible phrasings](#)
 - [5. The system prompt is a control, not the enforcement mechanism](#)
 - [6. Data-sharing consideration for a real deployment](#)
 - [7. Performance is undocumented](#)
 - [8. Provider migrated from Anthropic to Gemini, and an inherited Phase 6 ROI bug fixed, during post-delivery audit](#)
 - [9. Interactive chat CLI \(`chat.py` \) rebuilt from a fragile draft](#)

PHASE 10B — LIMITATIONS

Consolidated from the phase report, per Master Prompt Section 16.

1. No live Gemini API call was made in this build sandbox

No `GEMINI_API_KEY` was available in this environment. `ai_explanation_layer.py`'s `_call_gemini()` function is real, production-ready code (correct endpoint, headers, message format, model), but it was never actually invoked against the live API to produce this deliverable. The no-key fallback path was exercised for real; the LLM-response-handling path was tested with `unittest.mock.patch` substituting synthetic response text, disclosed directly in the test file's docstring. Anyone running this with a real key set exercises the same code path with no changes required — but live network behavior (latency, response format edge cases, rate limits) has not been observed firsthand.

2. The fidelity validator checks a fixed, explicit list of protected fields

`PROTECTED_FIELD_PATHS` covers the 10 fields §10.10 and the Decision Policy consider decision-critical: decision score, decision confidence, IRR, NPV, cap rate, ROI, DSCR, probability of negative NPV, probability of meeting target IRR, location score, and market strength score. It does not attempt to validate arbitrary other numbers an LLM might mention (a locality's population density, a specific infrastructure item's distance, etc.). If a future caller needs the same hard guarantee for additional fields, extending the list is the correct approach — the validator's logic doesn't need to change, only its field registry.

3. Contradiction detection depends on recognizable labeling

When a protected number doesn't appear in its correct numeric form, the validator falls back to checking whether the field's *topic* (its label, e.g. "IRR", "decision score") is discussed nearby, to distinguish "altered" from "simply not mentioned." An LLM response that discusses a concept using substantially different wording than the labels registered in `PROTECTED_FIELD_PATHS` — e.g., writing "the property's expected return" instead of "IRR" — could theoretically have an altered number misclassified as omitted rather than contradicted. The system prompt instructs the model to use standard financial terms, which mitigates this in practice, but this is a prompt-level mitigation, not a code-level guarantee, and is disclosed here rather than assumed away.

4. Currency-representation matching covers common forms, not all possible phrasings

`_representations()` for currency values accepts whole-rupee, lakh, crore, and million-scale numeric forms (e.g., “2847332”, “28.47” lakhs, “2.85” million). It does not parse number words (“two point eight five million”) or unusual formatting. A correctly-stated number in an unrecognized format could be misclassified as omitted rather than matched — this would not cause a false failure (omission isn’t a failure), but could understate the reported match rate in the fidelity report.

5. The system prompt is a control, not the enforcement mechanism

`SYSTEM_PROMPT` instructs the model not to alter numbers, invent figures, or override the decision — but Section 3 of the phase report is explicit that this instruction is not trusted by itself. The `numeric_fidelity_validator.py` code-level check is the actual enforcement; the prompt’s role is to reduce how often the validator has to intervene, not to replace it.

6. Data-sharing consideration for a real deployment

The full `DecisionResult` JSON — including property-level financial figures (asking price, IRR, NPV, cap rate) — is sent to Google’s Gemini API when the live LLM path is used. This is standard for any LLM-based explanation feature, but a production deployment should confirm this is acceptable under the business’s own data-handling and privacy policies before enabling `GEMINI_API_KEY` in a live environment.

7. Performance is undocumented

No live call was made, so no latency or throughput figures exist for this phase. A real deployment should expect standard Gemini API response times (typically low single-digit seconds for the ~1,200-token max response configured here) plus a negligible amount of time for the pure-Python fidelity check.

8. Provider migrated from Anthropic to Gemini, and an inherited Phase 6 ROI bug fixed, during post-delivery audit

This phase was originally built against Anthropic’s Messages API and has been migrated to Google’s Gemini `generateContent` REST endpoint per explicit direction (`_call_gemini()`, `GEMINI_API_KEY`, `GEMINI_MODEL`). `numeric_fidelity_validator.py` needed no changes – it validates text against a `DecisionResult` and has no provider awareness, which is the intended separation of concerns. Separately, the same review found `src/finance/engine.py` here was pulled in from Phase 6 *before* Phase 6’s own post-delivery audit fixed `roi_total_holding_period` (it was computing a gross cash multiple instead of a net return, overstating ROI by exactly 100 percentage points in every case – the same inherited-stale-copy issue independently found in Phases 7, 9, and 10A). This mattered more here than elsewhere: the fidelity validator lists ROI as a protected number, so it would have faithfully defended the wrong value against correction. **Fixed** by re-copying Phase 6’s corrected `engine.py`; one new regression test (`test_real_result_roi_protected_number_is_net_return_not_gross_multiple`) added to `test_phase10b.py` – suite is now 15/15 passing. See `docs/phase10b_report.md` Section 5B for full detail.

9. Interactive chat CLI (`chat.py`) rebuilt from a fragile draft

A user-supplied `chat.py` draft could only reliably do one thing (resolve *some* property id and explain it), and did that unreliably: it matched any bare 4-digit number as a `property_id` (misfiring on years/percentages), ranked “best/top” properties by raw CSV row order (unrelated to investment quality), had no path for general/conceptual questions, and read a `result.get('valid')` key that `generate_ai_explanation()` does not return (always silently printed `None`). Rebuilt as a proper query-routing pipeline (`src/chat/query_router.py`, `ranking.py`, `locality_lookup.py`, `general_qa.py`) – full design rationale, two real bugs found and fixed during the rebuild’s own testing (a module-name collision between `src/finance/batch_runner.py` and `src/geo/batch_runner.py`, and a ranking tie-break that let an AVOID property outrank INVEST properties on a “safest investment” query), and the new 29-test suite are documented in `docs/phase10b_report.md` Section 5C. The chat CLI still inherits every other limitation in this document (synthetic data, UNCALIBRATED risk figures, disclosed data gaps, `--source csv` re-check path) – it is a new way to ask questions of the same governed pipeline, not a new source of numbers.

